

High Throughput Large Integer Multipliers Built on Optimized FPGA DSP Block Bases

Abstract—Large integer multiplication is critical in cryptographic applications such as Fully Homomorphic Encryption (FHE) and RSA where it represents the main step in computing the modular multiplication used to generate keys. Most previous work building such multipliers on FPGAs has overlooked the importance of considering the underlying architecture of the Digital Signal Processing (DSP) Blocks present in modern FPGAs. These DSP blocks have an irregular input size that must be tiled carefully to create larger multipliers, and have complex pipelining configurations, which must be respected to achieve high frequency. We show how to build efficient base multipliers that can then be tiled to large sizes efficiently through automated architecture-aware tiling and fast pipelined datapath generation, while maintaining resource efficiency and high throughput. We demonstrate full automation of large multiplier generation, achieving 56% higher frequency while using about 17% fewer DSP blocks compared to prior approaches.

I. INTRODUCTION

While multiplication is conceptually simple, implementing it efficiently becomes difficult at large sizes. Large integer multiplication is especially important in cryptography, where common sizes include 160–512 bits for Elliptic Curve Cryptography (ECC) and 1024–4096 bits for RSA [1]. These large sizes must be built from the smaller primitives available on a target architecture.

Field-Programmable Gate Arrays (FPGAs) offer a flexible architecture that can be used to implement a variety of applications in domains such as image processing, digital signal processing [2], and cryptography [3]. Because multiplication is a common operation and expensive to implement in LUTs, FPGAs have provided dedicated DSP blocks that deliver higher throughput and better area/energy efficiency than LUT implementations. DSP blocks also support common arithmetic patterns (e.g. multiply-accumulate). However, achieving an optimal balance of DSPs and LUTs for large, structured arithmetic remains challenging.

Large multipliers are typically constructed by decomposing operands into smaller tiles that match the underlying DSP wordlengths. Schoolbook decomposition computes multiple partial products and combines them with shifts and additions, while Karatsuba decomposition [4] reduces the number of multiplication operations in exchange for additions/subtractions (Section II). For

extremely large sizes, FFT [5] and NTT [6] based methods are also possible.

Effective large multiplication decomposition on FPGAs must consider the underlying architecture to achieve high performance and resource efficiency: DSP primitives are asymmetric with non-power-of-2 wordlengths, signed/unsigned handling affects tiling, and achieving high frequency requires respecting DSP pipeline constraints. FPGAs also expose additional structures such as cascade paths that can improve both performance and resource usage when exploited. This paper addresses large integer multiplication in an architecture-aware manner, enabling both efficient base multipliers and scalable large multipliers on FPGAs.

The main contributions of this paper are:

- 1) A method for building architecture-aware base multipliers that map efficiently to FPGA DSP blocks while achieving high frequency and low resource usage, shown to outperform vendor 64-bit IP multiplier.
- 2) Automated construction of large multipliers from the proposed base designs using Schoolbook and Karatsuba decompositions, with FPGA-aware pipelining and mapping.
- 3) Evaluation across operand sizes and base sizes, including a 1024-bit-bit design compared against an optimized HLS implementation and the state-of-the-art approaches in [7] and [8].
- 4) An open-source tool for generating large multipliers on AMD FPGAs, which will be released with the paper for the wider community.

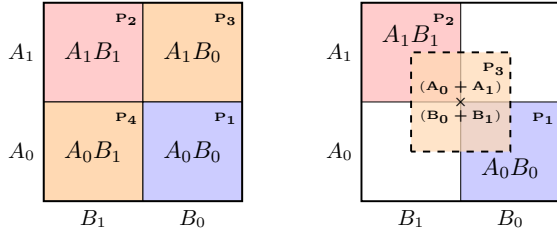
II. BACKGROUND

A. Decomposition Schemes

Large multipliers are typically built by decomposing the computation across smaller multipliers with shift-and-add operations combining these to form the final result. **Schoolbook Decomposition** forms a $2n \times 2n$ product using four $n \times n$ partial products (Equation 1). Let $A = (A_1 \ll n) + A_0$ and $B = (B_1 \ll n) + B_0$; with reference to Fig. 1a:

$$\text{Schoolbook}(A \times B) = P1 + (P2 \ll 2n) + ((P3 + P4) \ll n) \quad (1)$$

This approach scales to more splits (e.g., nine multipliers for a 3×3 split) and relies on more shift-and-add logic to



(a) Schoolbook decomposition. (b) Karatsuba decomposition.

Fig. 1: Comparison of Schoolbook and Karatsuba decomposition schemes for a $2n \times 2n$ multiplier.

combine partial products. Equation 1 can be rearranged to trade an addition for zero cost concatenation [7]:

$$(P1 + (P2 \ll 2n)) = \{P2, P1\} \quad (2)$$

Karatsuba Decomposition [4] reduces the number of multiplications needed in the 2×2 case from four to three by replacing one multiplication with additions/subtractions. One product is typically $(n+1)$ bits wide. Fig. 1 compares Schoolbook and Karatsuba for a $2n \times 2n$ multiplier. Karatsuba can be extended to larger grids (e.g., 3×3), saving three multipliers relative to Schoolbook (which uses 9 multipliers).

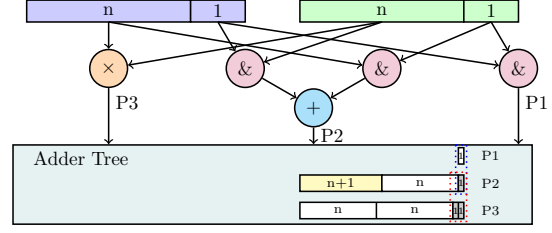
$$\text{Karatsuba}(A_0B_1 + A_1B_0) = P3 - P1 - P2 \quad (3)$$

The $(n+1)$ -bit multiply can be implemented using an n -bit multiplier plus additional logic by splitting each operand into an n -bit part and a 1-bit part (Fig. 2) to maximally use a fixed-size component multiplier for all products. The $n \times 1$ and 1×1 terms reduce to AND gates, and non-overlapping terms can be concatenated (Equation 2). Placing the $n \times n$ multiplier on the LSB side (Fig. 2b) reduces the widest adder from $2n$ to $n+1$ bits.

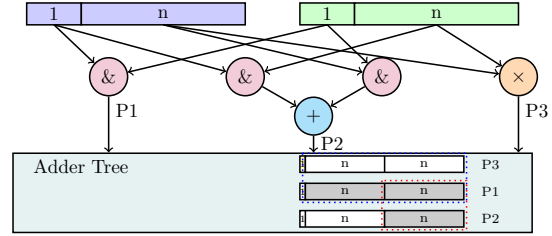
B. DSP Block Consideration

Overview and Applications. A DSP block is a hard FPGA primitive optimized for high-frequency multiply-add/accumulate operations. Modern architectures have increased multiplier widths (e.g., 25×18 in Virtex-5/6/7, 27×18 in UltraScale(+), and 27×24 in Versal). They are widely used in FIR filters, ML inference, and scientific signal processing [9], [10]; some generations also support floating-point modes [11].

AMD DSP Block Architecture. An AMD DSP48E2 block integrates a pre-adder (not shown nor used in this work), multiplier, multiplexers, and an ALU (Fig. 3). Control signals (e.g., OPMODE/ALUMODE) select the datapath/ALU expression [12], and optional pipeline registers trade latency for throughput (enabling all four



(a) The n -bit multiplier is used for the MSB.



(b) The n -bit multiplier is used for the LSB.

Fig. 2: $n+1$ -bit multiplier used in Karatsuba decomposition. The grey areas denote the shift amount (added zeros). The dotted rectangles denote concatenation. The LSB variant reduces the widest adder from $2n$ to $n+1$ bits.

registers yields maximum throughput). Cascaded ports (e.g., PCIN/PCOUT) support efficient column-wise chaining. Correct multiplier and cascade behavior requires routing the multiplier partial products to the X and Y multiplexers while selecting the appropriate Z source (e.g., PCIN) via OPMODE.

Chaining DSP Blocks. More than one DSP block can be chained to create wider multipliers. For instance, two DSP blocks can be chained to perform a 44×18 multiplication. Assuming two inputs K of size 44 bits and M of size 18 bits, K is split into two parts: the lower 17 bits (K_L) and the upper 27 bits (K_H). To fit within the 48-bit datapath and leverage the built-in 17-bit right shift, the accumulation is rewritten as follows:

$$P1 = (K_L \times M), \quad P2 = (K_H \times M) \quad (4)$$

$$K \times M = \{(P2 + (P1 \gg 17)), P1[16:0]\}$$

Fig. 3 contrasts two chaining options: the PCOUT-to-PCIN cascade path versus routing P-to-C. With PCOUT-to-PCIN, S1 cascades PCOUT to S2 PCIN and uses the built-in 17-bit right shift in S2. Signal synchronization requires delaying K_H , M , and the extracted LSBs from P1, resulting in a 5-cycle latency, but no additional hardware usage. Because the cascade path supports only 0 or 17-bit shifts, other shift amounts must use the P-to-C path which requires shift/sign-extension (ExSRn) and routing S1's P to S2's C, typically adding an extra pipeline stage for signal synchronization.

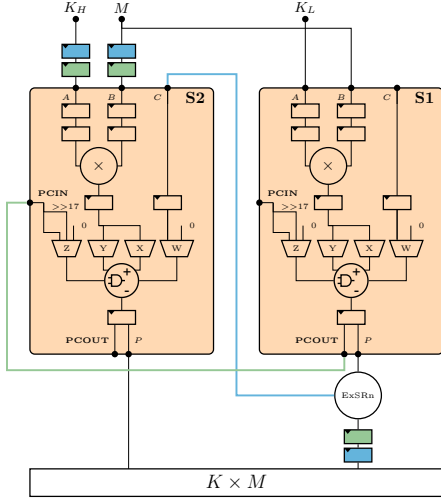


Fig. 3: Comparison of DSP48E2 cascade paths: PCOUT-to-PCIN (green) vs. P-to-C (blue). The ExSRn block extracts n bits from P to output while shifting P right by n bits, sign-extending, and routing the result to the C input. The P-to-C path requires additional logic and an extra pipeline stage.

C. Signed vs Unsigned Multiplication

When constructing large multipliers from smaller ones, signedness must be handled carefully. In decomposition, only the multipliers dealing with MSBs should be signed (for a signed multiplication); all other partial products must be computed unsigned even if the overall multiplication is signed. A signed n -bit multiplier can be used as an unsigned $(n-1)$ -bit multiplier by zero-padding the MSB.

D. IP Multipliers

AMD provides an IP multiplier in Vivado [13] that generates parameterized signed/unsigned multipliers up to 64×64 bits, with options to optimize for frequency or resource utilization and a recommended pipeline depth. For sizes larger than 47×47 bits, only the performance-optimal design is offered; it achieves maximum DSP frequency but consumes more DSP blocks than necessary.

III. LARGE MULTIPLIER DESIGN

Our approach focuses on building efficient base multipliers using DSP blocks, then hierarchically tiling them up to large sizes while considering the FPGA architecture and DSP block features and using fast pipelined combinational logic.

A. Building Base Multipliers

To construct large multipliers efficiently on FPGAs, we build base multipliers and tile them to larger sizes. Karat-

suba decomposition can only combine partial products that are aligned to the same shift amount. For a $W_1 \times W_2$ base, shifts take the form $W_1n + W_2m$ and Karatsuba-friendly shift amounts take the form $k \cdot \frac{W_1W_2}{\gcd(W_1, W_2)}$, where k is an integer. Thus, rectangular bases only yield sparse Karatsuba-friendly alignments (e.g., $26 \times 17 \Rightarrow 442k$). Square bases yield more Karatsuba-eligible partial products (e.g., $44 \times 44 \Rightarrow 44k$), allowing more opportunities for reducing the number of multipliers used.

A square base multiplier can be obtained using a tiling algorithm inspired by Roy et al. [14]. Given DSP primitive unsigned dimensions $W_1 \times W_2$ (e.g., 26×17 for the DSP48E2), and a target size M , the algorithm chooses integers n, m so that the covered width

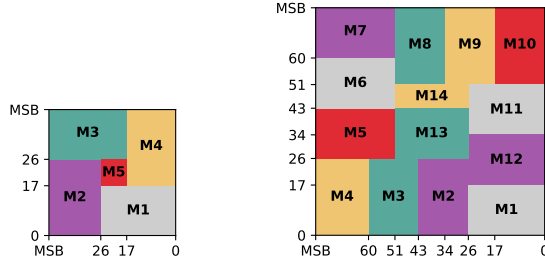
$$\bar{M} = n \cdot W_1 + m \cdot W_2.$$

where $\bar{M} \geq M$. Tiles of size $W_1 \times W_2$ and $W_2 \times W_1$ are placed along the boundary of the $\bar{M} \times \bar{M}$ square, leaving a smaller square region in the middle that is recursively tiled in the same way. However, the original algorithm has practical limitations that our work addresses:

- The remaining middle region is not guaranteed to be a square for arbitrary n, m .
- Hardware mapping is not architecture-aware (e.g., cascade/shift constraints), and does not explicitly provide variants that trade off LUT usage and latency.

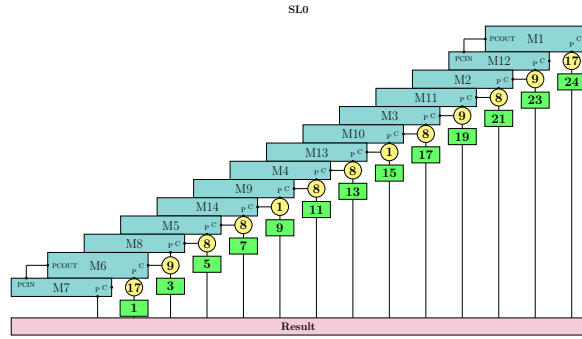
To guarantee a square middle region, we constrain one tiling coefficient to 1 (either $n = 1$ or $m = 1$). Fig. 4 shows the tiling scheme for 44- and 78-bit signed base multipliers. To address the remaining issues, we separate generating multiplication terms (tiles) from mapping them to DSP blocks. This enables architecture-aware mapping (cascade/shift constraints) and optimization for either LUT minimization or latency. Additionally, small multipliers are automatically mapped to LUTs to save DSP resources for larger widths, and all signals are automatically synchronized. Fig. 5 compares two mapping strategies for the 78-bit multiplier. First, the one-group strategy (Fig. 5a) chains as many terms as possible (sorted in ascending order based on their shift amount) to minimize LUT usage. This ordering allows for non-overlapping bit extraction and keeps additions within the DSP48E2 48-bit datapath. The multi-group strategy (Fig. 5b) forms independent chains whose terms are shifted apart by 17 bits, maximizing usage of cascade connections and enabling parallel evaluation to reduce latency.

Table I compares the resource usage and latency of the 44- and 78-bit signed base multipliers built using the two grouping strategies. The one-group strategy generally uses fewer LUTs, while the multi-group strategy reduces latency at the cost of slightly increased LUT usage for the adder tree. In the 78-bit one-group variant, increased

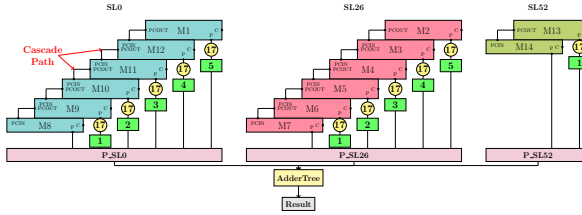


(a) 44-bit tiling (5 DSP48E2). (b) 78-bit tiling (14 DSP48E2).

Fig. 4: Tiling schemes for different square base multiplier sizes.



(a) One group strategy (minimizing LUTs).



(b) Multi-group strategy (minimizing latency).

Fig. 5: Grouping strategies for the 78-bit base multiplier. Green rectangles indicate pipeline stage counts. Yellow circles show the number of LSBs extracted from P and the corresponding right shift amount passed to C when using the P-to-C path. The multi-group strategy reduces latency, while the one-group strategy minimizes LUT usage.

LUT usage results from the synthesis tools using LUTs to synchronize datapath signals instead of FF registers.

To test the effectiveness of this base multiplier design, we compare against the AMD Vivado IP 64-bit unsigned multiplier discussed in Section II-D. We compare it against the closest size obtainable by our base tiling algorithm: a 77-bit unsigned multiplier (implemented by zero-padding the MSB of the 78-bit multiplier). Our base multiplier achieves a slightly higher frequency than the IP, supports larger inputs, and uses 2 fewer DSPs at the cost

TABLE I: Comparison of different base multiplier strategies.

Size	Primitive Strategy	LUTs	FFs	DSPs	Latency (Cycles)
44	One group	78	323	5	9
	Multi-group	105	334	5	8
78	One group	537	1051	14	28
	Multi-group	429	1386	14	14
64	AMD IP	219	722	16	18

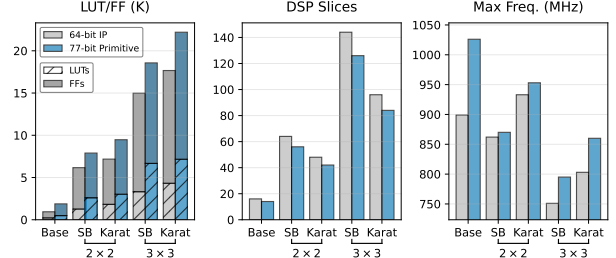


Fig. 6: Resource and frequency comparison between the 64-bit AMD IP and our 77-bit base multiplier when tiled to larger sizes.

of extra 210 LUTs. To measure DSP block utilization, we set the theoretical minimum number of DSP blocks to tile an $S \times S$ multiplier to be $\frac{S^2}{26 \times 17}$. The 64×64 IP exceeds this bound by 6.7, while our 77×77 design is within 0.59 block of it (only sacrifices a few bits in one block). Fig. 6 compares the two base designs when tiled to larger sizes using 2×2 and 3×3 Schoolbook and Karatsuba decompositions. When tiled at one level of hierarchy, our 77-bit base multiplier consistently uses fewer DSPs and comparable LUTs, while achieving a higher frequency.

B. Tiler

Finding the optimal placement of small rectangles to tile a large rectangle is NP-complete [15]. General tiling schemes suffer from two main limitations when applied to large multipliers on FPGAs:

- They do not prioritize hierarchical regions that enable Karatsuba-style factoring.
- They ignore architecture-specific features such as DSP cascade capabilities and efficient base sizes.

We address these limitations by hierarchically replicating an efficient square base multiplier of size $B \times B$ (Section III-A) using 2×2 or 3×3 grids that preserve shift alignment for systematic Karatsuba decomposition. Accordingly, the possible scaled up side lengths are

$$H = 2^x \times 3^y \times B,$$

where each factor of 2 or 3 corresponds to one hierarchy level.

The total DSP usage can be determined once the base and decomposition schedule are chosen. Let D_{base} be the DSP usage of the base multiplier, then for a hierarchical side length $H = 2^x \times 3^y \times B$, the total DSP utilization is:

$$D_{total} = D_{base} \times \prod_{i=1}^L \left(\frac{s_i \times (s_i + k_i)}{2^{k_i}} \right)$$

where $L = x + y$ is the total number of hierarchy levels. At each level i , $s_i \in \{2, 3\}$ for 2-way or 3-way split, and $k_i \in \{0, 1\}$ for Schoolbook or Karatsuba decomposition.

We evaluate two tiling methodologies, compared in Fig. 7 for a 1024-bit target multiplier:

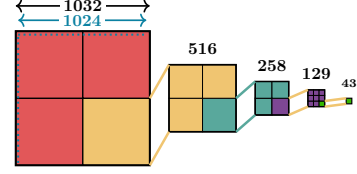
Overshoot tiling. Instead of forcing an exact target size match, we choose an achieved size $\bar{T} \geq T$ that increases the hierarchical portion (e.g., by selecting a larger H or using a base B with better $2^x \times 3^y$ coverage). This can provide more opportunities for Karatsuba decomposition and thus reduce DSP usage, at the cost of extra LUT-based shift-and-add logic to handle the excess bits.

Target-constrained tiling. Alternatively, given a target side length T , we first select the largest hierarchical side length $H \leq T$ of the form $H = 2^x \times 3^y \times B$. This produces a hierarchical $H \times H$ region that can be decomposed cleanly across hierarchy levels; the remaining non-hierarchical rectangular regions are then filled using additional placements.

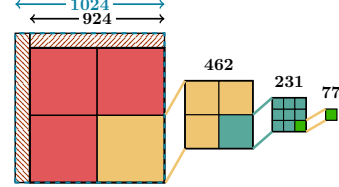
Tiling the remaining area. We explore three variants for this: (1) greedily placing the same base until it no longer fits, then placing DSP-sized multiplications; (2) recursively tiling the remainder using other efficient bases (e.g., 44×44 and 94×94) and small DSP-sized tiles; and (3) a cascade-compatible variant that prefers placements enabling DSP cascade connections. In all variants, each candidate placement is scored by DSP count and by estimated LUT/FF overhead for accumulation. The cascade-compatible variant is preferred in Fig. 7b because it reduces LUT-based shifting and addition while also lowering DSP utilization (464 DSP48E2 blocks versus 507 for the greedy approach and 516 for the non-cascade-compatible variant for the 1024-bit multiplier in Fig. 7).

C. Optimal Base Size Choice

Base multiplier size strongly affects DSP usage, hierarchy depth, and the amount of irregular remainder tiling (Section III-B). We evaluate the candidate bases produced in Section III-A. Bases derived by fixing the larger DSP dimension (26 for DSP48E2 26×17) consistently reduce DSP waste (e.g., 43, 60, 77, 94, 111, 128, $\dots$, 264). Within this family, the 77-bit base exhibits the lowest waste when considered in isolation. However, low DSP waste alone does not guarantee an efficient hierarchy for Karatsuba or Schoolbook decomposition. We therefore also consider nearby sizes that improve hierarchical



(a) Overshoot tiling with 43-bit base showing the final achieved size exceeds the target.



(b) Target-constrained tiling with 77-bit base, showing a remaining portion that must be tiled with other multipliers.

Fig. 7: Comparison of overshoot and target-constrained tiling for a 1024-bit multiplier. Overshoot tiling exceeds the target size but maximizes hierarchical structure. Target-constrained tiling stays within the target, filling the remainder with additional multipliers. Each level of the hierarchy can be implemented using Schoolbook or Karatsuba decomposition.

coverage, even if they yield a slightly larger achieved size. For example, tiling the 77-bit base towards a 1024-bit target yields a largest hierarchical block of 924-bits and leaves two non-hierarchical rectangular regions. In contrast, a 43-bit base can reach 1032-bits and thus increase the hierarchical nature for Karatsuba (Fig. 7).

Since DSP usage and latency are known at design time, we perform a Pareto analysis across targets from 160 to 1024 bits with different bases and decomposition schemes. Fig. 8 shows that the 43-bit base with Karatsuba decomposition at all levels minimizes DSP usage at the cost of higher latency, while 77- and 128-bit bases with Schoolbook decomposition reduce latency but increase DSP consumption.

D. Fast Pipelined Combinational Logic

The wide combinational paths implementing logic like shift/add/subtract can dominate the critical path in large multipliers. Therefore, we split the logic into smaller pipelined segments to achieve high throughput. Empirically, we observe a frequency drop when the segment width exceeds 64 bits. Our flow makes segment size configurable to trade off depth, latency, and timing. These intermediate logic segments are pipelined to achieve high throughput; this is done not only at the module level, but also with awareness of the pipeline schedule of individual signals, interleaving pipelining where necessary to reduce overall latency.

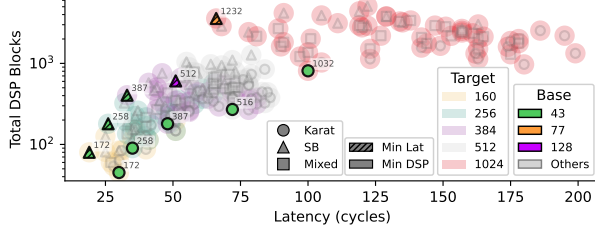


Fig. 8: DSP block usage and latency trade-offs for different base sizes across target multiplier sizes (log scale y-axis). Annotations indicate the achieved sizes. The 43-bit base with Karatsuba decomposition at all levels consistently minimizes DSP usage.

E. Automating the Flow

To enable rapid design-space exploration, we built an automated flow that integrates the components described in the previous sections. A specification is provided with the target multiplication size, base multiplier dimensions, option desired decomposition strategies (Schoolbook, Karatsuba, or mixed), and the base combinational logic width. The **Tiler** first instantiates a grid covering the total size using the base size, with no hierarchy. When the target size is not an exact multiple of the base, the remaining regions are filled using DSP block multipliers (Section III-B). A single hierarchy level exposes limited structure for Karatsuba-style factoring. Hence, the **Hierarchy Generator** takes this initial flat grid and generates alternative hierarchical combinations of 2×2 and 3×3 grids at different levels. For example, a flat 12×12 grid can be decomposed into a grid of four 6×6 blocks, each of which can be a grid of four 3×3 bases, creating three levels of hierarchy (see Fig. 7b).

Each variant is processed by the **Exporter**, and the **Visualizer** in parallel. The **Exporter** outputs an intermediate representation of the decomposition scheme that indicates the mathematical operations without timing consideration. The **Visualizer** renders the decomposition graphically and checks for correct tiling (100% coverage with no overlaps). The **Software Tester** then validates the untimed intermediate representation via software simulation before hardware generation. After validation, the **RTL Generator** takes the intermediate representation and produces timed, optimized, synthesizable Verilog for each module and submodule based on Section III-A and Section III-D, along with testbenches for functional verification. The flow also generates Tcl scripts to run FPGA implementation and report area/timing, and generates dataflow diagrams with annotated operations and pipeline synchronization points.

IV. EVALUATION

A. Experiments and Results

We target the AMD Alveo U280 FPGA (XCU280 part). We generate large multipliers for operand sizes from 160 to 1024 bits using overshoot tiling with a 43-bit base and target-constrained tiling with a 77-bit base as discussed in Section III-B. For the 1024-bit case, we also applied our method to the 64-bit AMD IP multiplier as a base in which overshoot and target-constrained approaches yield the same result (four levels of 2×2 grids). Wide additions are pipelined using 64-bit segments. We report frequency, LUTs, and DSPs obtained from Vivado implementation post place and route. We also report a normalized area metric based on the ratio of LUTs to DSPs on the target device (1082K LUTs and 8490 DSPs [16] for the U280) as used in [7], [17]. This ratio of 126 LUTs per DSP on the target device allows us to normalize area to a single metric as $\#LUTs + (126 \times \#DSPs)$.

We compare against **Impress** (an HLS framework that uses E-graph-based rewriting at different levels of the hierarchy to determine which decomposition to apply) [7] which provides two results with different tradeoffs, the HLS baseline reported in their paper, and the beam-search tiling heuristic from [8]. Table II lists the lowest DSP utilization configurations generated by our tool per target size and base; other configurations are omitted for brevity. Fig. 9 summarizes DSP and normalized-area trends across decomposition schedules for the 77-bit and 43-bit bases. Across targets, increasing the number of Karatsuba levels generally reduces DSP usage and normalized area, and often improves frequency. The 43-bit base benefits more from deeper Karatsuba hierarchies due to its larger number of hierarchical levels. The 77-bit base is typically competitive when fewer Karatsuba levels are used.

Our method achieves significantly higher frequency (and hence throughput) than previous work, regardless of the base used. Using the 64-bit IP multiplier as a base wastes resources, while our 77- and 43-bit base multipliers dramatically reduce DSP block usage. Fig. 10 compares the frequency of our implementations with other work against both DSP count and normalized area. The HLS design uses the fewest DSPs but has low frequency and high normalized area. **Impress** uses moderate resources but also runs at low frequency. Our 43-bit primitive-based design achieves the highest frequency while maintaining moderate DSP usage and normalized area. It achieves 83% higher frequency than **Impress1**, with 16% fewer LUTs and the same number of DSPs. Against **Impress2**, it achieves 56% higher frequency while using 17% fewer DSPs and slightly fewer LUTs. It also achieves 62% higher frequency than the beam-search tiling heuristic in [8], while reducing DSP block usage by 45% and LUTs by 5%. Our method is general

TABLE II: Detailed comparison of different decomposition schemes for various multiplier sizes. Target-constrained tiling is used with the 77-bit base multiplier, while overshoot tiling is used with the 43-bit base.

Target Size	Actual Size	Base	#Karat. Levels	LUTs	FFs	DSPs	Freq. (MHz)	Lat. (Cyc)
160	162	27×18	0	3461	6112	61	830	35
	160	77	1	3770	7507	62	855	31
	172	43	2	5141	9202	45	862	30
256	264	27×18	0	9947	15710	161	760	64
	256	77	1	8473	17718	114	776	40
	258	43	2	10858	20171	90	842	35
384	384	77	2	21350	38330	279	583	56
	387	43	2	23208	43803	180	751	48
512	512	77	2	33122	61525	370	619	72
	516	43	3	38131	68870	270	629	60
1024	1024	77	3	123472	207765	1220	478	123
	1032	43	4	128252	221612	810	458	100
1024	1024	IP	4	105717	169858	1296	459	89
Related Work								
1024	HLS [7]		6	165618	N/A	729	298	24
	IMpress1 [7]		N/A	150400	N/A	810	250	23
	IMpress2 [7]		N/A	129338	N/A	945	295	22
	BeamSearch [8]		N/A	133975	N/A	1471	283	N/A
Versal VCK190								
1024	1176	49	4	163712	274787	684	496	115

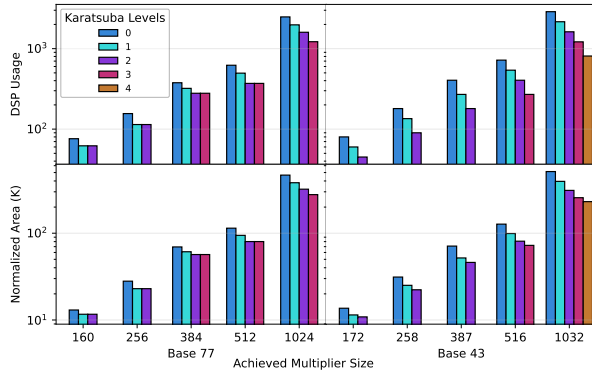


Fig. 9: Resource utilization for target-constrained tiling using our 77-bit base and overshoot tiling using our 43-bit base multipliers, with varying numbers of Karatsuba levels. 77-bit base is favorable for fewer Karatsuba levels, while 43-bit base benefits from deeper Karatsuba hierarchies. Note the logarithmic y-axis.

and not tied to a specific primitive dimension; we also provide a preliminary evaluation for the DSP58 primitives on the AMD Versal (VCK190 development board). We constructed a 49-bit unsigned base multiplier that requires five DSP58 blocks combined with a 3×3 multiplier using LUTs. This optimized base is then scaled using overshoot tiling, following the same approach as the DSP48E2 43-bit design. As shown in Table II, the DSP58-based multiplier significantly reduces DSP usage and improves frequency, at the cost of a slight increase in LUTs.

B. Latency Mitigation

As shown, our primitive-based large multipliers achieve significantly higher frequencies than prior work, but they

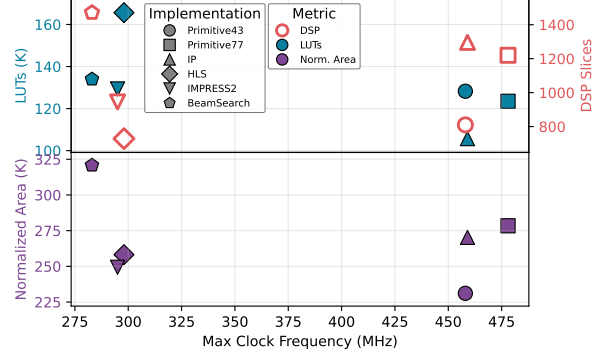


Fig. 10: Area and Performance comparison of different implementations. Our primitive-based implementation achieves approximately 56% higher frequency than the best among other work while using 17% fewer DSPs and slightly fewer LUTs.

TABLE III: Impact of combinational logic segment size on latency and performance for 1032-bit multiplier 4-Karatsuba with 43-bit base

Segment Size	LUTs	FFs	Freq. (MHz)	Lat. (Cycles)
96	127225	225670	379	88
128	125480	219590	359	70
192	133741	216455	368	66
256	136056	213833	348	58

do suffer from higher latency in cycles. This can be mitigated by divide the combinational logic into larger segment sizes (e.g., 128 bits instead of 64 bits). Table III shows the impact of varying the combinational logic segment size on the latency and performance of a 1032-bit multiplier with a 43-bit base multiplier. Increasing the segment size reduces the latency significantly, trading off slightly increased LUT usage and reduced maximum frequency. Using a 256-bit segment size reduces the latency from 100 to 58 cycles while still achieving higher frequency than other implementations. With the significant frequency improvement against previous work, this latency cost can be tolerable for the high throughput improvement.

V. RELATED WORK

Authors in [17] developed an automated tool for mapping arithmetic expressions to DSP blocks through DSP-aware pipelining. However, they relied on fixed template databases that constrain their generalizability and are not suitable for large integer multiplication. Authors in [7] presented IMpress, an E-graph framework for large integer multiplication using HLS. However, it suffers from low frequency. Extending general decomposition schemes to be architecture-aware has been explored in previous work. In [14], they proposed a general algorithm to tile

base multipliers previously obtained in [18]. However, the approach is limited to schoolbook decomposition, and is not suited to large numbers. Authors in [19] proposed a binary ILP formulation followed by beam search heuristic tiling [8] for integer multiplication. Both run at low frequency and are not area efficient in large sizes. Karatsuba-based tiling for DSP blocks has been explored in [20], but it is not scalable for bigger operands as it does not take advantage of hierarchical decomposition.

VI. CONCLUSIONS AND FUTURE WORK

Implementing large integer multiplication in FPGAs in a way that maximizes performance while also using resources efficiently is challenging. In this work, we developed an approach for building efficient square base multipliers suited to DSP block implementation, exploiting their architectural features. We automate the process of generating the RTL code for tiling these bases to build large-integer multipliers in an architecture-aware manner with fast pipelined combinational logic. The generated designs achieve high frequency, 56% higher than previous work, while using around 17% fewer DSP blocks. We are releasing the tool for use by the wider research community.

In the future, we plan to leverage equality saturation to generalize this flow to arbitrary arithmetic expressions, exploiting architecture-aware rewriting in a similar manner. Finally, for very large multipliers, we can develop a higher level pipeline structure for better scaling across SLRs.

VII. REFERENCES

- [1] D. Patel, B. Patel, J. Vasa, and M. Patel, “A comparison of the key size and security level of the ECC and RSA algorithms with a focus on cloud/fog computing,” in *ICT with Intelligent Applications*, 2023, pp. 43–53.
- [2] S. Memik, A. Katsaggelos, and M. Sarrafzadeh, “Analysis and FPGA implementation of image restoration under resource constraints,” *IEEE Transactions on Computers*, pp. 390–399, 2003.
- [3] T. Ye, S. R. Kuppannagari, R. Kannan, and V. K. Prasanna, “Performance modeling and FPGA acceleration of homomorphic encrypted convolution,” in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, 2021, pp. 115–121.
- [4] A. Karatsuba and Y. Ofman, “Multiplication of multidigit numbers on automata,” *Soviet Physics Doklady*, vol. 7, p. 595, 1962.
- [5] C. Yap and C. Li, “Quickmul: Practical FFT-based integer multiplication,” 2000.
- [6] N. Emmart and C. C. Weems, “High precision integer multiplication with a GPU using strassen’s algorithm with multiple FFT sizes,” *Parallel Processing Letters*, pp. 359–375, 2011.
- [7] E. Ustun, I. San, J. Yin, C. Yu, and Z. Zhang, “IMpress: Large integer multiplication expression rewriting for FPGA HLS,” in *Field-Programmable Custom Computing Machines (FCCM)*, 2022, pp. 1–10.
- [8] A. Böttcher, K. Kullmann, and M. Kumm, “Heuristics for the design of large multipliers for FPGAs,” in *IEEE Computer Arithmetic (ARITH)*, 2020, pp. 17–24.
- [9] Xilinx, Inc., “DSP: Designing for optimal results. high-performance DSP using virtex-4 FPGAs (UG073),” 2005.
- [10] D. Wang, K. Xu, J. Guo, and S. Ghiasi, “DSP-efficient hardware acceleration of convolutional neural network inference on FPGAs,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 4867–4880, 2020.
- [11] M. Langhammer and B. Pasca, “Floating-point DSP block architecture for FPGAs,” in *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2015, pp. 117–125.
- [12] Xilinx, Inc., “Ultrascale architecture DSP slice user guide (UG579),” 2021.
- [13] —, “Multiplier v12.0 LogiCORE IP product guide (PG108),” 2015.
- [14] D. B. Roy, D. Mukhopadhyay, M. Izumi, and J. Takahashi, “Tile before multiplication: An efficient strategy to optimize DSP multiplier for accelerating prime field ECC for NIST curves,” in *Design Automation Conference (DAC)*, 2014, pp. 1–6.
- [15] Beaumont, Boudet, Rastello, and Robert, “Partitioning a square into rectangles: NP-completeness and approximation algorithms,” *Algorithmica*, p. 217–239, 2002.
- [16] Xilinx, Inc., “Alveo U280 data center accelerator card data sheet(DS963),” 2019.
- [17] B. Ronak and S. A. Fahmy, “Mapping for maximum performance on FPGA DSP blocks,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 573–585, 2016.
- [18] F. Dinechin and B. Pasca, “Large multipliers with less DSP blocks,” in *Field Programmable Logic and Applications (FPL)*, 2009.
- [19] M. Kumm, J. Kappauf, M. Istioan, and P. Zipf, “Resource optimal design of large multipliers for FPGAs,” in *IEEE Computer Arithmetic (ARITH)*, 2017, pp. 131–138.
- [20] M. Kumm, O. Gustafsson, F. De Dinechin, J. Kappauf, and P. Zipf, “Karatsuba with rectangular multipliers for FPGAs,” in *Computer Arithmetic (ARITH)*, 2018, pp. 13–20.